React Developer Interview Assignment

Contact Personnel: Priti B, Rohan G

🔴🟠🟢

🎯 Assignment Title

Build an Interview Management Dashboard

📝 Objective

Create a frontend-only Interview Management Dashboard using **React** and **TypeScript**. Your implementation should demonstrate:

- Efficient performance
- UI-level security aligned with **OWASP Top 10**
- Modularity and reusability
- Clean and readable code structure
- Compliance with industry standards and frontend best practices

---

🛠️ Scope of Work

1. Authentication & Role Simulation

- Implement a login screen using the DummyJSON API `/auth/login`
- Simulate three roles: `admin`, `ta_member`, `panelist` (details below)
- Based on the role, redirect to the appropriate dashboard
- Store session securely using browser storage
- Implement logout functionality to clear the session and redirect to login

2. Dashboard View

- Display key performance metrics (KPIs):
  - Number of interviews scheduled this week
  - Average feedback score

    - Number of no-shows
- Provide filtering options for role, interviewer, and date range
- Present data using cards or charts, as appropriate

3. Candidate Management

- Fetch and display the candidate list using DummyJSON `/users`
- Show relevant fields like name, department, role, and interview status
- Support filtering, search, sorting, and pagination
- Provide actions like "View Details" and "Submit Feedback", shown conditionally based on role

4. Candidate Detail Page

- Include tabbed sections:
    - Profile: Basic info and resume
    - Schedule: List of tasks/interviews using `/todos?userId=<id>`
    - Feedback: View or submit feedback using `/posts?userId=<id>`
- Restrict tab visibility and actions based on role

5. Feedback Form

- Include input fields for:
    - Overall Score
    - Strengths
    - Areas for Improvement
- Validate all inputs and prevent form submission until valid
- Only panelists should see and submit the feedback form

6. Role Management (Admin Only)

- Provide a mock interface to manage user roles and permissions
- Hide/disable UI elements for users who don't have access
- Simulate access enforcement purely on the frontend

🔐 UI Security Expectations (OWASP Top 10)

Implement UI-level security based on **OWASP Top 10 for Web Applications**:

- **Broken Access Control**: Do not show unauthorized actions or navigation options
- **Cryptographic Failures**: Avoid storing passwords or sensitive session data insecurely
- **Injection**: Sanitize and validate all user input in forms and filters
- **Security Misconfiguration**: Do not expose debug logs or hardcoded secrets
- **Authentication Issues**: Handle login/logout reliably; never assume trust
- **Software/Data Integrity Failures**: Only use well-known and trusted UI libraries
- **Logging and Monitoring Failures**: Log only development-safe and non-sensitive information
- **CSRF Awareness**: Disable buttons after submission and provide visual feedback

You are not expected to build backend security — only simulate safe frontend behaviors.

🔌 API Integration

Use only the **DummyJSON API**: https://dummyjson.com

Key endpoints:

- `POST /auth/login`: simulate login
- `GET /users`: fetch candidate list
- `GET /users/:id`: candidate details
- `GET /todos?userId=<id>`: interview tasks
- `GET /posts?userId=<id>`: submitted feedback

Use fields like `company`, `tags`, or `department` to simulate real-world logic.

🧩 Simulating Roles with DummyJSON

Since DummyJSON does not support backend role definitions, roles must be **simulated on the frontend** using below method:

User Selects Role on Login

Let the user select a role from a dropdown on the login page (for testing flexibility).

- Store the selected role in localStorage or session state
- Useful if you want to quickly switch behavior for testing UI access controls

After assigning a role using any of the above strategies:

- Use the stored role to enforce route-level or component-level access control
- Conditionally show/hide elements based on the role

💡 Component Libraries & Tools

To accelerate development, you are encouraged to use **ready-to-use component libraries or styling frameworks** of your choice. Some options include:

- **StencilJS** – for reusable Web Components
- **Shadcn/UI** – for modern accessible components
- **Material UI** – for enterprise-grade UI design
- **TailwindCSS** – for utility-first styling and responsiveness
- **Storybook** – for documenting and testing UI components

You may use any combination. Document your choices in the `README.md`.

🧰 Tech Stack Requirements

Mandatory

- **Next.js** with **TypeScript**
- Functional React components with hooks
- App routing using **Next.js pages or app router**
- State management using **Context API** or **Redux Toolkit**
- At least one UI/styling framework:
    - TailwindCSS

    - Material UI
    - Shadcn/UI
    - StencilJS
- Use of **React Hook Form** or **Formik** for form management

Optional but Encouraged:

- Lazy loading and code splitting
- Unit testing with React Testing Library

📦 Deliverables

- GitHub repository with:
    - Setup instructions in `README.md`
    - Explanation of folder structure and tech choices
    - Role and permission matrix
    - OWASP Top 10 UI security coverage (as comments or section in README)
    - Optional: Storybook or test coverage notes

🧪 Evaluation Criteria

Area	Assessed On
Performance	Pagination, debounce, lazy loading
Defensive Programming	Input validation, fallback states
UI Security	Compliance with OWASP Top 10 (UI side)

Modularity	Component separation, reusability
Readability	Naming consistency, clear structure
Industry Standards	Code conventions, typing, accessibility
Best Practices	See next section
Clean Code	No dead code, duplication, or over-complex logic

📘 Frontend Development Best Practices

- **Component Modularity**: Break large UIs into reusable components
- **Folder Structure**: Organize code by feature or domain
- **Type Safety**: Define and use interfaces for all API responses and props
- **Secure Session Handling**: Store only non-sensitive data in local/session storage
- **State Management Discipline**: Use global state only when necessary
- **Accessibility (A11y)**: Use semantic HTML, keyboard navigation, and ARIA labels
- **Responsive Design**: Ensure layouts work across mobile, tablet, and desktop
- **Error Handling**: Display loading, error, and empty states gracefully
- **Performance Optimization**: Avoid unnecessary re-renders and large prop trees
- **Code Quality**: Maintain consistent formatting, naming, and structure

---

🏁 Final Note

This project simulates a real internal dashboard used by talent and engineering teams. Treat it like production code.

Design with clarity. Code with purpose. Demonstrate frontend discipline and security awareness.

Good luck!

End of Document